9-Hour Hands-on Java Training: Library Management System

Target audience: Second-year UG Computer Science students

Role: Java Trainer

Duration: 9 hours

Training approach: Learn the concept → demonstrate a small program → implement a feature in the Library Management System → test the feature.

The best way to conduct this training is to develop one complete, console-based Java project throughout the nine hours. Students will learn Java fundamentals while building a practical application rather than studying each topic independently.

1. Training objectives

By the end of the training, students should be able to:

- Understand Java program structure, variables, data types, operators, control statements, methods, classes and objects.
- Apply encapsulation, constructors, collections and exception handling in a real application.
- Use **Lambda expressions** to filter, search and sort library records.
- Use the **Java Date and Time API** to record borrowing dates, calculate due dates and identify overdue books.
- Understand the Java Platform Module System (JPMS) and create a modular Java application.
- Develop, execute and test a Library Management System using a step-by-step implementation approach.

2. Nine-hour training schedule

The schedule below assumes nine instructional hours, excluding any separate lunch break.

1 hr — Hour 1

Session 1 — Java fundamentals and project setup

- JDK, JRE, JVM, compilation and execution
- `main()` method, variables, data types and operators
- `if`, `switch`, loops and console input
- Create the project and display the library menu

Deliverable: Running menu-driven Java application.

1 hr — Hour 2

Session 2 — Classes, objects and encapsulation

- Define `Book` and `Borrower` classes
- Fields, constructors, getters and setters
- Access modifiers and object creation

- Display book and borrower details

Deliverable: Book and borrower data models.

1 hr

Hour 3

Session 3 — Collections, methods and exception handling

- `ArrayList`, `List` and `Map`
- Add, view and search books
- Store borrowers and transactions
- Validate input and handle invalid operations

Deliverable: Book catalogue and borrower registry.

1 hr

Hour 4

Session 4 — Borrowing, returning and business rules

- Issue a book to a borrower
- Prevent issuing an unavailable book
- Return a book and update availability
- Create and track transaction records

Deliverable: Working circulation workflow.

1 hr

Hour 5

Session 5 — Java Date and Time API

- `LocalDate`, `Period` and `ChronoUnit`
- Borrow date, due date and return date
- Date arithmetic and overdue calculation
- Display overdue transactions

Deliverable: Date-aware borrowing and overdue reports.

1 hr

Hour 6

Session 6 — Lambda expressions

- Functional interfaces and `Predicate`
- Lambda syntax: `(parameters) -> expression`
- Filter books by title, author or availability
- Sort books using `Comparator` and lambdas

Deliverable: Search, filter and sort features.

1 hr

Hour 7

Session 7 — Java Modules (JPMS)

- Why modules are useful
- `module-info.java`

- `exports` and `requires`
- Separate model, service and application modules

Deliverable: Modularized library application.

1 hr

Hour 8

Session 8 — Integration and complete application testing

- Integrate all modules and features
- Test valid and invalid inputs
- Verify issue, return and overdue workflows
- Correct compilation and runtime errors

Deliverable: Integrated, tested application.

1 hr

Hour 9

Session 9 — Student hands-on challenge and evaluation

- Add one new feature independently
- Run test cases and demonstrate the application
- Explain lambdas, dates and modules
- Review code quality and possible enhancements

Deliverable: Demonstrated project and individual assessment.

3. Project requirements and feature list

Project title: University Library Management System

The university wants a simple application to manage its books, borrowers and borrowing transactions. For a nine-hour training session, use a console application with in-memory collections. This keeps the focus on Java rather than database configuration or UI development.

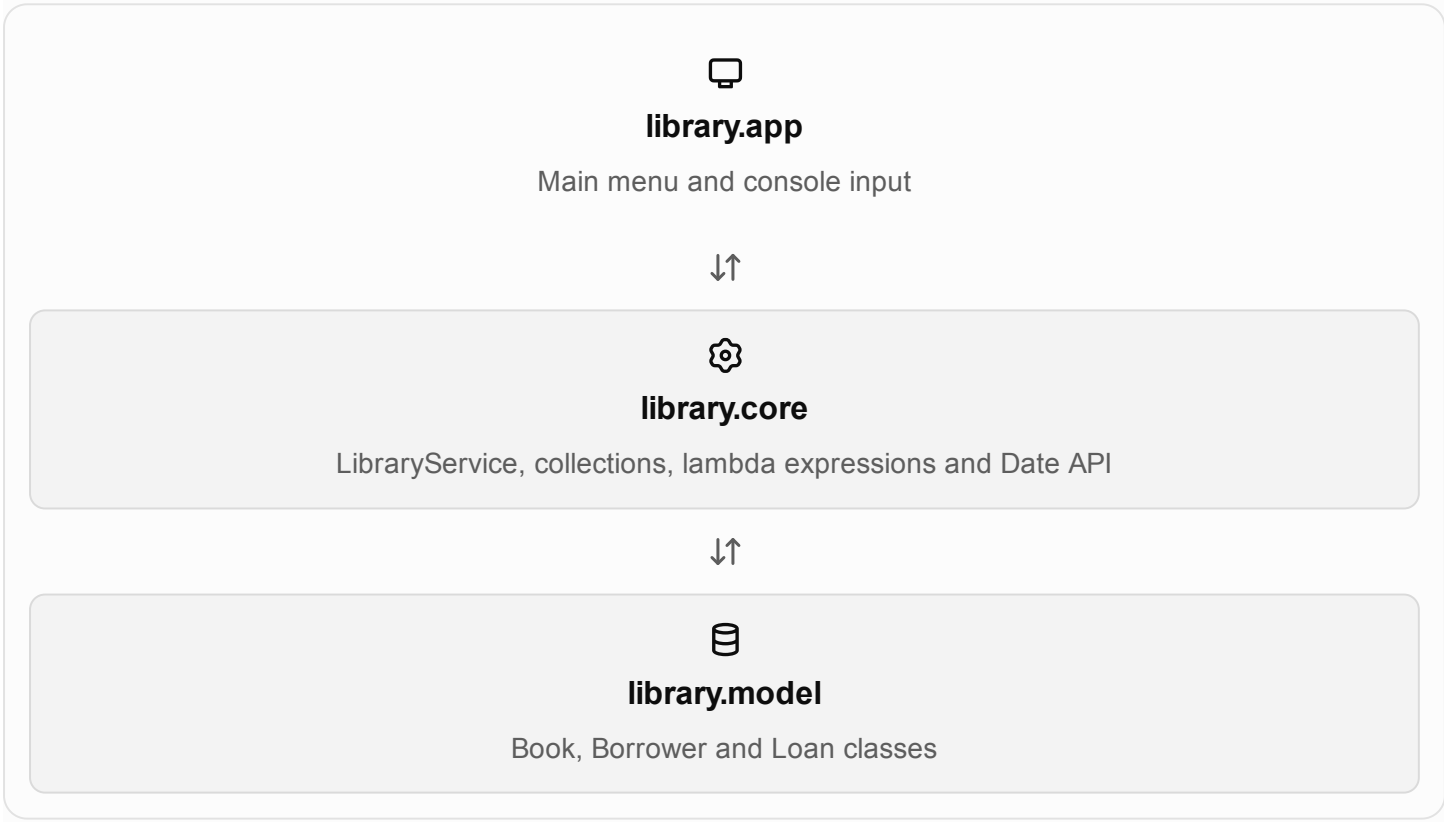
Module	Features students implement
Book management	Add, view, search and sort books
Borrower management	Register and view borrowers
Issue management	Issue available books and calculate due dates
Return management	Record returns and update availability
Transaction management	View loan history and current loans
Overdue report	Identify books past their due date
Java enhancements	Lambda expressions, Date API and JPMS modules

Business rules

1. Every book has a unique ID, title, author and availability status.
2. Every borrower has a unique ID, name and email.
3. Only available books can be issued.
4. A loan is assigned a borrowing date and a due date 14 days later.
5. A returned book becomes available again.
6. A loan is overdue when it has not been returned and the current date is after its due date.
7. Searching should be case-insensitive, and books can be sorted by title.
8. Invalid book IDs, borrower IDs and duplicate registrations must be handled appropriately.

4. Project architecture

Use **Java 17 or later** so the students can use the module system and modern Java features.



This three-module structure demonstrates JPMS clearly. The project uses collections for temporary storage; data will be lost when the application exits. Persistent storage can be a later extension.

5. Complete Java project source code

Create the following folder structure:

Plain text

```
LibraryManagementSystem/
└── src/
```

```
├── library.model/
│   ├── module-info.java
│   └── com/university/library/model/
│       ├── Book.java
│       ├── Borrower.java
│       └── Loan.java
├── library.core/
│   ├── module-info.java
│   └── com/university/library/service/
│       └── LibraryService.java
└── library.app/
    ├── module-info.java
    └── com/university/library/app/
        └── Main.java
```

Module 1: `library.model`

File: `src/library.model/module-info.java`

Java

```java
module library.model {
    exports com.university.library.model;
}
```

This exports the model package so other modules can use its public classes.

File: `Book.java`

Java

```java
package com.university.library.model;

public class Book {
    private final int id;
    private final String title;
    private final String author;
    private boolean available = true;

    public Book(int id, String title, String author) {
        this.id = id;
        this.title = title;
        this.author = author;
    }

    public int getId() {
```

```
        return id;
    }

    public String getTitle() {
        return title;
```

File: `Borrower.java`

Java

```
package com.university.library.model;

public class Borrower {
    private final int id;
    private final String name;
    private final String email;

    public Borrower(int id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }
```

File: `Loan.java`

Java

```
package com.university.library.model;

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

public class Loan {
    private final int id;
    private final Book book;
    private final Borrower borrower;
    private final LocalDate borrowDate;
    private final LocalDate dueDate;
    private LocalDate returnDate;
```

```
    public Loan(int id, Book book, Borrower borrower,
                LocalDate borrowDate, LocalDate dueDate) {
        this.id = id;
        this.book = book;
        this.borrower = borrower;
        this.borrowDate = borrowDate;
        this.dueDate = dueDate;
```

Module 2: `library.core`

File: `src/library.core/module-info.java`

Java

```
module library.core {
    requires library.model;

    exports com.university.library.service;
}
```

File: `LibraryService.java`

This class demonstrates `ArrayList`, `HashMap`, `LocalDate`, lambda expressions, `Predicate` and `Comparator`.

Java

```
package com.university.library.service;

import com.university.library.model.Book;
import com.university.library.model.Borrower;
import com.university.library.model.Loan;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.function.Predicate;
import java.util.stream.Collectors;

public class LibraryService {
    private final Map<Integer, Book> books = new HashMap<>();
```

```
    private final Map<Integer, Borrower> borrowers = new HashMap<>();
    private final List<Loan> loans = new ArrayList<>();
```

Module 3: `library.app`

File: `src/library.app/module-info.java`

Java

```
module library.app {
    requires library.model;
    requires library.core;
}
```

File: `Main.java`

Java

```
package com.university.library.app;

import com.university.library.model.Book;
import com.university.library.model.Borrower;
import com.university.library.model.Loan;
import com.university.library.service.LibraryService;

import java.util.List;
import java.util.Scanner;

public class Main {
    private static final Scanner scanner = new Scanner(System.in);
    private static final LibraryService library = new LibraryService();

    public static void main(String[] args) {
        boolean running = true;

        while (running) {
            printMenu();
```

6. Compile and run the modular project

Open a terminal in the `LibraryManagementSystem` directory. Ensure that JDK 17 or later is installed.

Check the Java installation:

```bash
java -version
javac -version
```

Compile all three modules:

```bash
javac -d out --module-source-path src \
  -m library.model,library.core,library.app
```

Run the application:

```bash
java --module-path out \
  -m library.app/com.university.library.app.Main
```

These commands work in a shell that supports multiline commands with backslashes, such as Linux, macOS or Git Bash on Windows. In Windows Command Prompt, enter each command on one line.

Sample test session

Expected application output

```
===== UNIVERSITY LIBRARY =====
1. Add Book
2. List Books
3. Search Books
4. Register Borrower
5. List Borrowers
6. Issue Book
7. Return Book
8. View Loan History
9. View Overdue Loans
0. Exit
```

```
Enter choice: 6
Book ID: 101
Borrower ID: 1
Book issued successfully.
Borrow date: 2026-10-12
Due date: 2026-10-26
```

The dates shown are illustrative for the current date. Actual output depends on the system date.

Try issuing book `101` a second time. The application should reject the request because the book is already issued. Return book `101`, then list the books again to verify that its availability has changed.

7. Detailed trainer delivery plan

Use the following practical sequence so students progressively build the application.

Hour 1 — Fundamentals

Explain: JDK, JVM, source code, bytecode, `main()`, variables, operators, conditions and loops.

Demonstrate: A small menu using `Scanner` and `switch`.

Student activity: Print the library menu and repeatedly accept choices until Exit is selected.

Check understanding: Ask students to explain why a loop is needed in a menu-driven program.

Hour 2 — Object-oriented Java

Explain: Classes, objects, fields, constructors, methods, access modifiers and encapsulation.

Demonstrate: Create a `Book` object and print its details.

Student activity: Create `Book` and `Borrower` classes, then instantiate three books.

Check understanding: Ask why fields should be private and why constructors are useful.

Hour 3 — Collections and validation

Explain: Lists, maps, generics and exceptions.

Demonstrate: Store books using a `Map<Integer, Book>` and prevent duplicate IDs.

Student activity: Implement adding, listing and searching books.

Check understanding: Ask why a map is useful when each book has a unique ID.

Hour 4 — Transactions

Explain: Relationships between objects and validation of business rules.

Demonstrate: Issue a book, mark it unavailable and create a loan.

Student activity: Implement issuing, returning and loan-history display.

Check understanding: Test issuing an unavailable book and returning a book without an active loan.

Hour 5 — Date API

Explain: Immutable dates, `plusDays()`, `isAfter()` and `ChronoUnit.DAYS.between()`.

Demonstrate:

Java

```java
LocalDate borrowDate = LocalDate.now();
LocalDate dueDate = borrowDate.plusDays(14);

System.out.println("Borrowed: " + borrowDate);
System.out.println("Due: " + dueDate);
```

Student activity: Add due dates and overdue calculations to the loan workflow.

Check understanding: Ask students to calculate the due date for a book borrowed on a specified date.

Hour 6 — Lambda expressions

Explain: Functional interfaces, lambda syntax, method references, streams, `filter()` and `sorted()`.

Demonstrate:

Java

```java
books.stream()
     .filter(book -> book.isAvailable())
     .sorted(Comparator.comparing(Book::getTitle))
     .forEach(System.out::println);
```

Student activity: Implement title/author search, available-book filtering and alphabetical sorting.

Check understanding: Ask students to explain what the lambda expression passed to `filter()` does.

Hour 7 — Java modules

Explain: Modules, packages, exported APIs and dependencies.

Demonstrate: Create three `module-info.java` files and compile the application.

Student activity: Separate the models, services and console application into modules.

Check understanding: Ask what happens if `library.core` does not declare `requires library.model`.

Hour 8 — Integration and testing

Explain: Integration testing, invalid input and regression testing.

Student activity: Run all menu operations and correct errors.

Trainer checklist: Verify duplicate IDs, invalid references, unavailable books, valid returns and overdue calculations.

Hour 9 — Assessment

Students complete an independent extension, demonstrate their code and explain the concepts used.

Suggested challenge: Add a report that counts how many books are available and how many are issued, using streams and lambda expressions.

8. Java concepts mapped to the project

Topic	Where students apply it
Variables and data types	Book IDs, titles, availability
Conditions and loops	Menu and business-rule validation
Classes and objects	`Book`, `Borrower`, `Loan`
Encapsulation	Private fields and public methods
Collections and generics	`List<Book>`, `Map<Integer, Book>`
Exception handling	Invalid IDs, duplicates and unavailable books
Lambda expressions	Search, filter, sort and overdue reports

Java Date API	Borrowing, due and return dates
Java modules	`module-info.java`, `requires`, `exports`

One important teaching point: **Lambda expressions were introduced in Java 8, the Date and Time API was also introduced in Java 8, and the Java Platform Module System was introduced in Java 9.** They are newer Java features relative to the original Java versions, but they are standard features in modern Java.

9. Student assessment rubric

Assessment criterion	Marks
Java fundamentals and object-oriented design	20
Book and borrower management	15
Issue and return transactions	20
Date API and overdue calculations	15
Lambda expressions and sorting/filtering	15
Java modules and compilation	10
Demonstration and code explanation	5
Total	100

10. Suggested student assignments

1. Add a feature to search borrowers by name.
2. Sort the book list by author.
3. Display only available books.
4. Add a report of books currently issued to each borrower.
5. Display the number of days remaining until a book's due date.
6. Modify the overdue report to sort transactions by the number of overdue days.
7. Add a maximum borrowing limit of three active books per borrower.
8. Add a fine calculation of ₹5 per overdue day, as a university-defined business rule.

Trainer recommendation: Distribute the project skeleton before training, but ask students to implement one feature at a time. Use Hours 1–4 to build the core application, Hours 5–7 to introduce the requested modern Java features, Hour 8 for integration, and Hour 9 for independent implementation and assessment. This makes the nine-hour session practical and achievable for second-year UG students.